

PROMPT MASTERY · 15 MIN READ

Advanced Prompt Engineering & Cognitive Reasoning Patterns

Mastering Chain-of-Thought, ReAct, Structured Decoding, and Defense

Executive Architecture Summary: The executive reference for production prompt engineering. Covers few-shot exemplar curation, Chain-of-Thought (CoT), ReAct decision loops, Tree of Thoughts (ToT), constrained JSON decoding, and prompt injection defense.

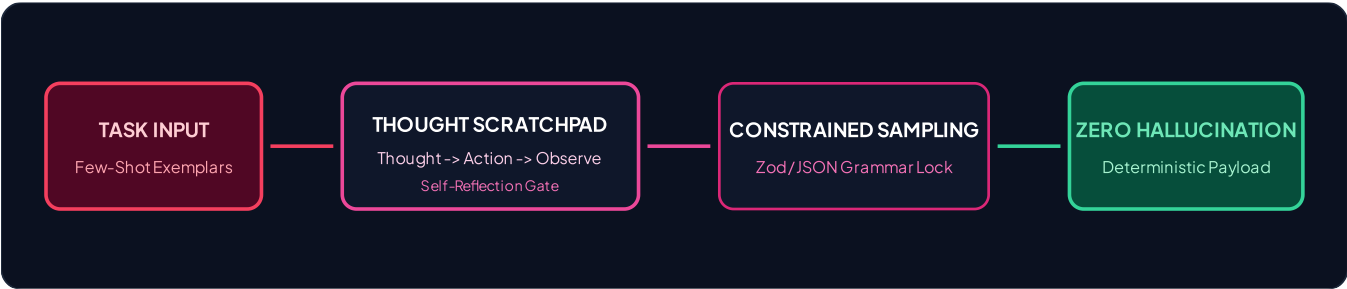
1. The Cognitive Shift: From Instructions to Thought Engines

Prompt engineering is not informal chatting; it is deterministic programming over probabilistic weights. Treating LLM context windows as virtual computers with instruction pointers, registers (system prompt), and working memory yields reliable software systems.

- Zero-Shot vs Few-Shot: Providing 3–5 high-variance input-output exemplars increases task accuracy by up to 48%.
- Chain-of-Thought (CoT): Forcing intermediate reasoning scratchpads prevents premature conclusion hallucination.
- Role Grounding: Specifying precise behavioral constraints and failure handling routines.
- Delimiters & XML Tags: Wrap inputs in `<input>`, `</input>`, and `<output>` to prevent boundary confusion.

2. Cognitive Reasoning Topologies

Select the appropriate reasoning pattern based on task complexity. Linear tasks use simple Chain-of-Thought, while uncertain planning problems require branching exploration.



3. Constrained Decoding & Guaranteed JSON Schema

Production applications cannot tolerate syntax errors in generated outputs. Enforce grammar-based constrained sampling or JSON mode with Zod / Pydantic validation.

```
// Production Structured Prompt Contract
const SYSTEM_DIRECTIVE = `
You are an Enterprise Lead Scoring Analyst. Analyze the prospect and output STRICT JSON only.
NO markdown code blocks, NO preamble, NO post-text.

JSON SCHEMA:
{
  "qualified": boolean,
  "confidenceScore": number (0-100),
  "painPoints": string[],
  "recommendedAction": "immediate_outreach" | "nurture_newsletter" | "disqualify",
  "reasoningDigest": string
}
`;

// Validate output through Zod Schema on arrival
const LeadScoreSchema = z.object({
  qualified: z.boolean(),
  confidenceScore: z.number().min(0).max(100),
  painPoints: z.array(z.string()),
  recommendedAction: z.enum(['immediate_outreach', 'nurture_newsletter', 'disqualify']),
  reasoningDigest: z.string()
});
```

- Temperature Management: Set temperature to 0.0 for deterministic extraction and code generation.
- Fallback Repair Loops: If JSON parsing fails, pipe error back to model in a single-turn repair prompt.

4. Security: Defending Against Prompt Injection

Untrusted user inputs and scraped web data can contain malicious prompt injections designed to hijack model behavior. Implement defensive input sanitization and dual-model arbitration.

- Input Tag Sandboxing: Treat all external text as untrusted data within strict XML boundaries.
- Constitutional Guards: Independent validation agent checks final outputs against policy violations before display.